Lecture 12: Video Generation, 3D Generation, and Physical AI

생성 모델의 응용 확장 — 비디오, 3D, 로봇 행동 생성

목차 (Table of Contents)

- 0. 개요 (Overview)
  - 0.1 선수 지식
  - 0.2 학습 목표
  - 0.3 응용 분야 요약
- Part A: 비디오 생성 (Video Generation)
  - A.1 이미지 → 비디오: 무엇이 달라지는가?
  - A.2 시간적 일관성 (Temporal Consistency)
  - A.3 비디오 DiT 구조
  - A.4 PyTorch 구현: 3D Patchify & Temporal Attention
  - A.5 주요 비디오 생성 모델
  - A.6 비디오 VAE
  - A.7 비디오 생성의 핵심 도전
- Part B: 3D 생성 (3D Generation)
  - B.1 2D Diffusion으로 3D를 만드는 방법
  - B.2 Score Distillation Sampling (SDS)
  - B.3 PyTorch 구현: Simplified SDS Loss
  - B.4 3D 생성 파이프라인 (DreamFusion)
  - B.5 후속 모델
  - B.6 Multi-View Diffusion
- Part C: Physical AI — Diffusion for Robotics
  - C.1 왜 Diffusion이 로봇에 필요한가?
  - C.2 Diffusion Policy
  - C.3 PyTorch 구현: Diffusion Policy 학습 및 추론
  - C.4 Diffusion Policy의 장점
  - C.5 World Models
  - C.6 PyTorch 구현: Simplified World Model
  - C.7 HPT (Heterogeneous Pre-trained Transformers)
- Part D: 기타 응용 (Other Applications)
  - D.1 과학: AlphaFold3
  - D.2 음악 생성
  - D.3 분자 설계

- 전체 시리즈 요약 (Full Series Summary)
- 참고문헌 (References)

---

0. 개요 (Overview)

강의 7~11에서 이미지 생성의 핵심 기술을 모두 배웠다. 이 강의에서는 이 기술들이 이미지를 넘어 **비디오, 3D, 로봇**으로 확장되는 방법을 다룬다.

> 💡 **왜 이미지를 넘어서야 하는가?** 이미지 생성은 2D 정적 콘텐츠에 국한된다. 그러나 현실 세계는 본질적으로 **시간적**(비디오), **3차원적**(3D), **물리적**(로봇 상호작용)이다. 이미지 생성에서 얻은 핵심 통찰 — "노이즈에서 데이터로의 반복적 정제" — 은 놀랍도록 범용적인 프레임워크여서, 최소한의 구조적 변경만으로 이 세 영역 모두에 적용할 수 있다. 이것이 Diffusion 모델이 단순한 이미지 생성 도구를 넘어 **범용 생성 프레임워크(universal generative framework)**로 자리잡게 된 핵심 이유다. 이 강의는 "같은 수학적 원리가 어떻게 서로 다른 도메인으로 전이되는가"를 이해하는 것이 핵심이다.

0.1 선수 지식

이 강의를 이해하기 위해 다음 개념들을 숙지해야 한다:

선수 지식	강의	이 강의에서의 활용
DDPM (Denoising Diffusion Probabilistic Models)	강의 7	Diffusion Policy의 기반, 비디오 노이즈 제거
Flow Matching	강의 10	Sora, Movie Gen 등 최신 비디오 모델의 학습 방법
DiT (Diffusion Transformer)	강의 11	비디오 DiT로 확장, Spacetime Patch 임베딩
LDM / Stable Diffusion	강의 9	잠재 공간에서의 생성, VAE 인코더/디코더 구조
CLIP & CFG	강의 8	텍스트 조건부 생성, Classifier-Free Guidance

0.2 학습 목표

이 강의를 마치면 다음을 이해할 수 있다:

1. 이미지 Diffusion 모델을 **비디오**로 확장할 때 추가되는 구조적 변경 (Temporal Attention, 3D Patchify, Video VAE)
2. 2D Diffusion 모델의 지식을 활용하여 **3D 객체를 생성**하는 SDS(Score Distillation Sampling) 기법
3. Diffusion 모델을 **로봇 행동 생성**에 적용하는 Diffusion Policy의 원리

4. **World Models**의 개념과 비디오 예측을 통한 행동 계획
5. Diffusion 기반 생성 모델의 **과학, 음악, 분자 설계** 등 다양한 응용

0.3 응용 분야 요약

응용	핵심 확장	대표 모델
비디오 생성	2D → 3D (시간 축 추가)	Sora, Movie Gen, VEO
3D 생성	2D Diffusion으로 3D 최적화	DreamFusion, Magic3D
Physical AI	행동을 확률 분포로 모델링	Diffusion Policy, HPT

> 💡 **세 응용 분야의 공통 원리** 세 분야 모두 동일한 핵심 패턴을 따른다: (1) 생성하려는 대상(비디오 프레임, 3D 표현, 로봇 행동)을 **연속 벡터 공간**으로 표현하고, (2) 그 공간에서 **노이즈 → 데이터 변환**을 학습하며, (3) 도메인 특화 조건(텍스트, 카메라 각도, 관측)을 주입한다. 차이점은 오직 (a) 데이터의 차원과 구조, (b) 조건의 종류, (c) 도메인 특화 구조적 귀납적 편향(inductive bias)뿐이다. 이 강의에서는 각 분야에서 이 세 가지가 구체적으로 어떻게 달라지는지를 중심으로 다룬다.

Part A: 비디오 생성 (Video Generation)

A.1 이미지 → 비디오: 무엇이 달라지는가?

> 💡 **왜 비디오 생성이 이미지 생성보다 근본적으로 어려운가?** 얼핏 보면 비디오는 "이미지를 여러 장 생성하면 되는 것"처럼 보인다. 하지만 이것은 양적 차이가 아니라 **질적 차이**다. 구체적으로 두 가지 이유가 있다:
>
> **첫째, 차원의 폭발(Dimensionality Explosion):**
>
> - 512x512 이미지 = 262,144 픽셀
> - 512x512 x 30fps x 5초 비디오 = 39,321,600 픽셀 (약 **150배**)
> - 이것은 단순히 "더 많은 데이터"가 아니다. Self-attention의 계산 복잡도는 토큰 수의 제곱에 비례하므로, 모든 픽셀에 full attention을 적용하면 $O(N^2)$에서 N이 150배 증가하여 **22,500배** 더 많은 계산이 필요하다. 이것이 factorized attention (spatial + temporal 분리)이 필수적인 이유다.
>
> **둘째, 시간적 일관성은 질적으로 새로운 도전:**
>
> - 각 프레임이 독립적으로 아름다워도 충분하지 않다. 인접 프레임 간에 **물체가 순간이동하지 않고**, **조명이 깜빡이지 않으며**, **물리 법칙을 암묵적으로 따라야** 한다.
> - 이것은 모델이 "예쁜 그림 그리기"를 넘어 **물리적 세계의 동역학(dynamics)을 이해**해야 한다는 뜻이다. 공이 위로 올라갔다가 내려오고, 물이 아래로 흐르며, 그림자가 광원의 방향에 따라 움직여야 한다.

비디오 = 이미지의 시퀀스. 핵심 추가 요소:

	이미지 생성	비디오 생성
데이터 차원	$H \times W \times C$	$T \times H \times W \times C$
일관성	공간적 일관성	공간적 + **시간적 일관성**
계산량	$O(HW)$	$O(THW)$ (10~100배)
새로운 도전	—	물리 법칙, 카메라 이동, 3D 일관성

⚠️ **핵심 차이**: 비디오 생성에서는 단순히 프레임 수가 늘어나는 것이 아니라, **프레임 간의 시간적 일관성(temporal coherence)**을 보장해야 한다. 이것이 비디오 생성을 이미지 생성보다 근본적으로 어렵게 만드는 요인이다.

구체적 수치 비교 — 왜 나이브한 접근이 불가능한가:

```
이미지 DiT (512×512):
  패치 수 = (512/16)² = 1,024 토큰
  Self-attention 비용 = O(1,024²) = ~1M 연산

비디오 DiT (512×512×150프레임, 나이브):
  패치 수 = (150/2) × (512/16)² = 75 × 1,024 = 76,800 토큰
  Full self-attention = O(76,800²) = ~5.9B 연산 (5,900배!)

비디오 DiT (Factorized attention):
  Spatial: O(T × (H'×W')²) = 75 × 1,024² = ~78M
  Temporal: O(H'×W' × T²) = 1,024 × 75² = ~5.8M
  합계: ~84M (나이브 대비 ~70배 절약)
```

이 수치가 보여주듯, factorized attention 없이는 비디오 DiT는 실용적으로 학습이 불가능하다.

A.2 시간적 일관성 (Temporal Consistency)

인접 프레임이 부드럽게 연결되어야 한다. 각 프레임을 독립적으로 생성하면:

```
프레임 1: 고양이가 왼쪽을 봄
프레임 2: 고양이가 오른쪽을 봄 (불연속!)
프레임 3: 다른 고양이가 나타남 (붕괴!)
```

해결: **Temporal Attention** — 프레임 간 관계를 학습.

⚠️ **왜 독립 생성이 실패하는가?** 이미지 DiT에서 각 프레임을 독립적으로 생성하면, 프레임마다 서로 다른 노이즈에서 출발하기 때문에 동일한 객체의 위치, 형태, 색상이 프레임마다 달라진다. Temporal Attention은 서로 다른 프레임의 동일 공간 위치에 있는 토큰들이 정보를 교환하게 하여 이 문제를 해결한다.

> 💡 **왜 3D Convolution만으로는 부족한가?** 시간적 일관성을 위한 가장 직관적인 해법은 **3D Convolution**이다. 2D Conv가 (H, W) 공간을 처리하듯, 3D Conv는 (T, H, W) 시공간을 처리한다. 하지만 3D Conv에는 치명적인 한계가 있다:
>
> - **제한된 수용 영역(receptive field)**: 커널 크기가 $3 \times 3 \times 3$인 3D Conv는 시간 축으로 3프레임만 볼 수 있다. 30fps 비디오에서 이것은 0.1초에 불과하다. "공이 날아가서 5초 후에 착지한다"는 장거리 시간 의존성(long-range temporal dependency)을 포착할 수 없다.
> - **카메라 컷, 장면 전환**: 영상에서 장면이 전환되거나 물체가 장애물 뒤로 사라졌다 다시 나타나는 경우, 비연속적인 시간 관계를 이해해야 한다. 이것은 지역적(local) 연산인 convolution으로는 원리적으로 불가능하다.
> - **Temporal Attention은 전역적(global)**: Self-attention은 시퀀스 내 모든 위치 쌍을 직접 비교하므로, 프레임 1의 정보가 프레임 150의 생성에 직접 영향을 미칠 수 있다. 이것이 long-range temporal coherence를 가능하게 하는 핵심이다.
>
> 실제로 최신 비디오 모델들은 **3D Conv + Temporal Attention을 함께** 사용한다. 3D Conv가 지역적 시간 패턴(짧은 움직임)을 효율적으로 포착하고, Temporal Attention이 전역적 시간 일관성(장기 서사)을 보장하는 상보적 역할을 한다.

이미지 Diffusion → 비디오 Diffusion 확장의 구체적 방법 (Inflation):

기존에 잘 학습된 이미지 Diffusion 모델(예: Stable Diffusion)을 비디오로 확장하는 가장 실용적인 방법은 **모델 inflation**이다:

```
단계 1: 사전학습된 2D U-Net / DiT를 가져온다
단계 2: 각 2D spatial layer 뒤에 temporal layer를 삽입한다
  - 2D Conv 뒤에 → 1D temporal Conv (시간 축만 처리)
  - 2D Spatial Attention 뒤에 → 1D Temporal Attention 삽입
단계 3: 새로 추가된 temporal layer만 학습 (기존 spatial layer는 고정 또
```

> 💡 **왜 이미지 모델에서 시작하는가 (Fine-tuning의 이유)?** 처음부터 비디오 모델을 학습하면 "사과가 빨갛다", "하늘이 파랗다" 같은 기본적인 시각 지식부터 다시 학습해야 한다. 이미지 모델은 이미 수십억 장의 이미지에서 이런 지식을 학습했으므로, 비디오 모델은 **시간적 동역학(temporal dynamics)만 추가로 학습**하면 된다. 이것은 학습 데이터와 계산량을 극적으로 줄여준다. 비유하자면, 사진을 잘 찍는 사람에게 영상 촬영을 가르치는 것이 처음부터 영상을 가르치는 것보다 훨씬 쉬운 것과 같다.

A.3 비디오 DiT 구조

> 💡 **왜 이런 구조(Factorized Attention)가 필요한가?** 비디오의 모든 시공간 토큰에 full 3D self-attention을 적용하면, 복잡도가 $O(T^2 \cdot H'^2 \cdot W'^2)$가 되어 계산이 불가능하다. 예를 들어 $T' = 75, H' = 32, W' = 32$이면 총 토큰 수는 76,800개이고, attention matrix는 $76,800 \times 76,800 \approx 59$억 개의 원소를 가진다. 이것은 단일 GPU 메모리에 담을 수 없다.

> **Factorized attention**은 이 문제를 "공간과 시간을 분리"하여 해결한다. Spatial attention은 각 프레임 내에서 독립적으로 수행하고($O(T \cdot (H'W')^2)$), Temporal attention은 각 공간 위치에서 독립적으로 수행한다($O(H'W' \cdot T^2)$). 합치면 $O(T \cdot (H'W')^2 + H'W' \cdot T^2)$로, full attention의 $O((T \cdot H'W')^2)$보다 극적으로 작다. 수학적으로 이것은 full attention의 **근사(approximation)**이지만, 실험적으로 비디오의 시공간 구조를 충분히 포착한다는 것이 검증되었다.

```
비디오 잠재 표현: [B, C, T, H, W]
        ↓
[3D Patchify]
  T×H×W → (T/t_p)×(H/h_p)×(W/w_p)개의 3D 패치
  각 패치 → 토큰
        ↓
[Video DiT Block × N]
  ├─ Spatial Self-Attention (같은 프레임 내)
  ├─ Temporal Self-Attention (같은 위치, 다른 프레임)
  ├─ Cross-Attention (텍스트 조건)
  └─ MLP
        ↓
[Unpatchify + VAE Decoder]
        ↓
비디오
```

Temporal Attention의 동작 원리를 단계별로 이해하기:

Temporal Attention은 수학적으로는 표준 self-attention과 동일하지만, **어떤 토큰들 사이에서** attention을 수행하느냐가 핵심이다:

```
전체 토큰 형태: [B, T', H', W', D]  (T'=시간, H'×W'=공간, D=차원)

Step 1: Reshape하여 시간 축이 시퀀스 길이가 되도록 변환
  [B, T', H', W', D] → [B×H'×W', T', D]
  의미: 각 공간 위치 (h, w)를 독립적인 "배치"로 취급

Step 2: 표준 self-attention 적용 (시퀀스 길이 = T')
  각 (h,w) 위치에서, 그 위치의 프레임 1, 프레임 2, ..., 프레임 T'의
  토큰들이 서로 attend. "이 위치에서 시간에 따라 무엇이 변하는가?"를 학습

Step 3: 원래 형태로 복원
  [B×H'×W', T', D] → [B, T', H', W', D]
```

$$\text{Temporal-Attn}(Q_t, K_t, V_t) = \text{softmax}\left(\frac{Q_t K_t^\top}{\sqrt{d}}\right) V_t$$

여기서 Q_t, K_t, V_t는 **같은 공간 위치, 다른 시간**의 토큰들로 구성.

> **직관적 이해**: 화면의 특정 위치 (h, w)에 있는 토큰이 "나는 프레임 5에서 빨간 공이야. 프레임 3에서 이 위치에 뭐가 있었지? 프레임 7에서는 뭐가 올 거지?"라고 다른 프레임들에 질문하는 것이다. 이를 통해 시간에 따른 변화 패턴(움직임, 변형, 출현/소멸)을 학습한다.

⚠️ **Spatial vs Temporal Attention의 차원 재배열**: Spatial Attention에서는 토큰을 `(B*T, H'*W', D)` 형태로 재배열하여 각 프레임 내에서 독립적으로 self-attention을 수행한다. Temporal Attention에서는 토큰을 `(B*H'*W', T', D)` 형태로 재배열하여 같은 공간 위치의 토큰들이 시간 축을 따라 attend한다.

Spatial + Temporal의 교대 적용이 작동하는 이유:

Spatial attention만으로는 각 프레임이 독립적으로 처리되어 시간 일관성이 없다. Temporal attention만으로는 각 공간 위치가 독립적으로 처리되어 프레임 내 공간 구조가 깨진다. 이 둘을 **교대로 적용**하면, 정보가 공간→시간→공간→시간으로 흘러가면서 시공간 전체에 걸친 일관성이 자연스럽게 구축된다. N개의 블록을 쌓으면, 정보는 점진적으로 더 넓은 시공간 범위로 전파된다. 이것은 마치 그래프에서 메시지 패싱(message passing)을 반복하면 모든 노드 간에 정보가 도달하는 것과 같은 원리다.

A.4 PyTorch 구현: 3D Patchify & Temporal Attention

3D Patchify — 비디오를 3D 패치 토큰으로 변환

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class VideoPatchEmbed3D(nn.Module):
    """비디오 잠재 표현을 3D 패치 토큰으로 변환하는 모듈.

    이미지 DiT의 PatchEmbed를 시간 축까지 확장한 것이다.
    비디오 [B, C, T, H, W] → 토큰 시퀀스 [B, N_tokens, D]
    """
    def __init__(
        self,
        in_channels: int = 4,        # VAE 잠재 채널 수
        embed_dim: int = 1152,        # Transformer 히든 차원
        patch_size_t: int = 2,        # 시간 축 패치 크기
        patch_size_h: int = 2,        # 높이 패치 크기
        patch_size_w: int = 2,        # 너비 패치 크기
    ):
        super().__init__()
        self.patch_size = (patch_size_t, patch_size_h, patch_size_w)

        # 3D 컨볼루션으로 패치를 임베딩 (stride = patch_size → 비중첩
        self.proj = nn.Conv3d(
            in_channels, embed_dim,
            kernel_size=self.patch_size,
            stride=self.patch_size,     # 비중첩(non-overlapping) 패
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Args:
            x: [B, C, T, H, W] 비디오 잠재 표현
        Returns:
            tokens: [B, N_t * N_h * N_w, D] 토큰 시퀀스
            grid_size: (N_t, N_h, N_w) 패치 그리드 크기 (unpatchify에
        """
        B, C, T, H, W = x.shape

        # 3D Conv → [B, D, N_t, N_h, N_w]
        x = self.proj(x)
        N_t, N_h, N_w = x.shape[2], x.shape[3], x.shape[4]

        # 토큰 시퀀스로 변환: [B, D, N_t, N_h, N_w] → [B, N_t*N_h*N_w
        x = x.flatten(2).transpose(1, 2)

        return x, (N_t, N_h, N_w)


# --- 사용 예시 ---
if __name__ == "__main__":
    # 비디오 잠재 표현: 배치 2, 채널 4, 16프레임, 32x32 해상도
    video_latent = torch.randn(2, 4, 16, 32, 32)
```

```
patch_embed = VideoPatchEmbed3D(
    in_channels=4, embed_dim=1152,
    patch_size_t=2, patch_size_h=2, patch_size_w=2
)
tokens, grid = patch_embed(video_latent)

print(f"입력 비디오 잠재: {video_latent.shape}")         # [2, 4,
print(f"출력 토큰:      {tokens.shape}")                # [2, 8*
print(f"패치 그리드:     {grid}")                        # (8, 1
```

Temporal Attention Block — 프레임 간 시간적 관계 학습

```python
class TemporalAttention(nn.Module):
    """같은 공간 위치의 토큰들이 시간 축을 따라 self-attention을 수행.

    핵심: 토큰을 (B*N_h*N_w, N_t, D) 형태로 재배열하여
    각 공간 위치에서 시간 축만의 attention을 수행한다.
    """
    def __init__(self, dim: int, num_heads: int = 16, qkv_bias: boo
        super().__init__()
        self.num_heads = num_heads
        self.head_dim = dim // num_heads
        self.scale = self.head_dim ** -0.5

        # Q, K, V를 한번에 생성
        self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
        self.proj = nn.Linear(dim, dim)
        self.norm = nn.LayerNorm(dim)

    def forward(
        self,
        x: torch.Tensor,          # [B, N_t * N_h * N_w, D]
        grid_size: tuple,          # (N_t, N_h, N_w)
    ) -> torch.Tensor:
        """
        1. 토큰을 시간 축 기준으로 재배열
        2. 같은 (h, w) 위치의 토큰들끼리 temporal self-attention
        3. 원래 형태로 복원
        """
        B, N, D = x.shape
        N_t, N_h, N_w = grid_size

        residual = x
        x = self.norm(x)

        # 재배열: [B, N_t*N_h*N_w, D] → [B, N_t, N_h, N_w, D]
        x = x.view(B, N_t, N_h, N_w, D)

        # 시간 축이 시퀀스가 되도록: [B, N_h, N_w, N_t, D] → [B*N_h*N_
        x = x.permute(0, 2, 3, 1, 4).reshape(B * N_h * N_w, N_t, D)

        # QKV 생성
        qkv = self.qkv(x).reshape(B * N_h * N_w, N_t, 3, self.num_he
        qkv = qkv.permute(2, 0, 3, 1, 4)  # [3, B*N_h*N_w, heads, N_
        q, k, v = qkv.unbind(0)

        # Scaled Dot-Product Attention (시간 축)
        attn = (q @ k.transpose(-2, -1)) * self.scale   # [B*N_h*N_
        attn = attn.softmax(dim=-1)

        x = (attn @ v)                                  # [B*N_h*
        x = x.transpose(1, 2).reshape(B * N_h * N_w, N_t, D)
        x = self.proj(x)
```

```python
        # 원래 형태로 복원: [B*N_h*N_w, N_t, D] → [B, N_t*N_h*N_w, D]
        x = x.reshape(B, N_h, N_w, N_t, D)
        x = x.permute(0, 3, 1, 2, 4).reshape(B, N, D)

        return residual + x


class VideoDiTBlock(nn.Module):
    """비디오 DiT의 단일 블록.

    순서: Spatial Attention → Temporal Attention → Cross-Attention →
    이미지 DiT 블록에 Temporal Attention이 추가된 형태.
    """
    def __init__(self, dim: int = 1152, num_heads: int = 16, mlp_ra
        super().__init__()

        # 1) Spatial Self-Attention (같은 프레임 내의 토큰 간)
        self.spatial_norm = nn.LayerNorm(dim)
        self.spatial_attn = nn.MultiheadAttention(dim, num_heads, ba

        # 2) Temporal Self-Attention (같은 위치, 다른 프레임 간)
        self.temporal_attn = TemporalAttention(dim, num_heads)

        # 3) Cross-Attention (텍스트 조건)
        self.cross_norm = nn.LayerNorm(dim)
        self.cross_attn = nn.MultiheadAttention(dim, num_heads, bat

        # 4) MLP (Feed-Forward)
        self.mlp_norm = nn.LayerNorm(dim)
        mlp_hidden = int(dim * mlp_ratio)
        self.mlp = nn.Sequential(
            nn.Linear(dim, mlp_hidden),
            nn.GELU(),
            nn.Linear(mlp_hidden, dim),
        )

    def forward(
        self,
        x: torch.Tensor,          # [B, N_t*N_h*N_w, D] 비디오 토큰
        context: torch.Tensor,     # [B, L, D] 텍스트 임베딩
        grid_size: tuple,          # (N_t, N_h, N_w)
    ) -> torch.Tensor:
        B, N, D = x.shape
        N_t, N_h, N_w = grid_size

        # --- 1) Spatial Self-Attention ---
        # 각 프레임을 독립적으로 처리: [B*N_t, N_h*N_w, D]
        x_s = self.spatial_norm(x)
        x_s = x_s.view(B, N_t, N_h * N_w, D).reshape(B * N_t, N_h *
        spatial_out, _ = self.spatial_attn(x_s, x_s, x_s)
        spatial_out = spatial_out.reshape(B, N_t, N_h * N_w, D).view
        x = x + spatial_out
```

```
        x = x + spatial_out

        # --- 2) Temporal Self-Attention ---
        x = self.temporal_attn(x, grid_size)

        # --- 3) Cross-Attention (텍스트 조건) ---
        x_c = self.cross_norm(x)
        cross_out, _ = self.cross_attn(x_c, context, context)
        x = x + cross_out

        # --- 4) MLP ---
        x = x + self.mlp(self.mlp_norm(x))

        return x
```

A.5 주요 비디오 생성 모델

Sora (OpenAI, 2024.02)

📄 *OpenAI, "Video generation models as world simulators", Technical Report, 2024*

```
텍스트 → [T5 등 텍스트 인코더] → 조건
          ↓
노이즈 [T×H×W] → [Spacetime DiT + Flow Matching] → 깨끗한 잠재 표현
          ↓
[Temporal VAE Decoder] → 비디오
```

핵심 특징:

- **Spacetime Patches**: 비디오를 3D 패치($t_p \times h_p \times w_p$)로 분할
- **가변 해상도/길이**: 다양한 종횡비와 길이를 동일 모델로 처리
- **장시간 일관성**: 최대 1분 비디오

⚠️ **Sora의 혁신**: 기존 비디오 모델이 고정된 해상도/프레임 수를 사용한 것과 달리, Sora는 **가변 길이 토큰 시퀀스**를 처리하여 다양한 종횡비와 길이의 비디오를 단일 모델로 생성한다. 이는 이미지 DiT에서 가변 해상도를 처리하던 방식의 자연스러운 확장이다.

> 💡 **Sora는 "세계 시뮬레이터"인가?** OpenAI는 Sora를 "world simulator"라고 불렀다. 이에 대해 두 가지 상반된 시각이 있다:
>
> **낙관적 시각**: Sora는 수백만 시간의 비디오 데이터에서 학습하여 중력, 관성, 충돌, 반사 등 물리 법칙의 **암묵적 모델(implicit model)**을 내재화했다. 비디오를 생성한다는 것은 미래 프레임을 예측한다는 뜻이고, 이것은 본질적으로 세계의 동역학을 이해하는 것이다.
>
> **회의적 시각 (Yann LeCun 등)**: Sora가 학습한 것은 **"비디오가 어떻게 보이는가"의 통계적 패턴**이지, 물리 법칙 자체가 아니다. 학습 데이터에 없는 상황(예: 달에서의 물리)에서는 정확한 예측을 할 수 없다. 또한 Sora는 여전히 물리적으로 불가능한

> 비디오(물체가 갑자기 사라지거나, 액체가 위로 흐르는 등)를 생성하곤 한다. LeCun은 진정한 world model은 pixel space가 아닌 **abstract representation space**에서 작동해야 한다고 주장한다.
>
> 현재의 합의는: Sora는 놀라운 비디오 생성 품질을 보이지만, "진정한 물리 이해"와 "통계적 패턴 매칭" 사이의 경계는 아직 불분명하다는 것이다.

Movie Gen (Meta, 2024.10)

📄 *Polyak et al., "Movie Gen: A Cast of Media Foundation Models", Meta Technical Report, 2024*

- **30B 파라미터** 비디오 모델
- Flow Matching + DiT 기반
- 텍스트→비디오 + 이미지→비디오 + 비디오 편집
- 16초, 768×768, 16fps

VEO (Google, 2024~2025)

📄 *Google DeepMind, "VEO 2 / VEO 3", 2024–2025*

- 대화형 비디오 생성
- 오디오와 비디오 동시 생성 (VEO-3)

A.6 비디오 VAE

> 💡 **왜 비디오 VAE가 필요한가?** 비디오 Diffusion을 픽셀 공간에서 직접 수행하면 차원이 너무 크다. 512x512x150프레임x3채널 = 약 1.2억 차원이다. 이미지 생성에서 Latent Diffusion (LDM)이 성공한 것과 같은 이유로, 비디오도 **잠재 공간에서 Diffusion**을 수행해야 한다. 비디오 VAE는 시공간 모두를 압축하여 Diffusion이 다루어야 할 차원을 극적으로 줄인다.

이미지 VAE를 시간 축으로 확장:

```
이미지 VAE:  [H, W, C]    → [h, w, c]        (공간 압축)
비디오 VAE:  [T, H, W, C] → [t, h, w, c]     (시공간 압축)

구체적 수치 예시:
  입력 비디오: [64, 512, 512, 3]
  공간 압축 8x: H,W → H/8, W/8
  시간 압축 4x: T → T/4
  잠재 채널: 4 (또는 16)
  출력 잠재: [16, 64, 64, 4]

  압축률: (64×512×512×3) / (16×64×64×4) = 50,331,648 / 262,144 ≈ 192
```

시간 축 압축률 = 4~8. 64 프레임 비디오 → 8~16 프레임의 잠재 표현.

⚠️ **비디오 VAE의 시간 압축**: 시간 축을 압축할 때 인접 프레임 간의 중복 정보를 제거한다. 예를 들어 4x 시간 압축은 4프레임마다 1개의 잠재 프레임을 생성하며, 이는 프레임 간 변화가 작은 비디오에서 매우 효율적이다. Temporal Causal Convolution을 사용하여 미래 프레임 정보가 현재에 누출되지 않도록 한다.

Temporal Causal Convolution이란?

일반적인 temporal convolution은 미래 프레임의 정보도 사용한다 (양방향). 하지만 비디오 생성에서 이것은 문제가 될 수 있다 — 프레임 t의 잠재 표현이 프레임 $t+1, t+2, \ldots$의 정보에 의존하면, **자기회귀적 생성(autoregressive generation)**이 불가능해진다. Causal convolution은 패딩을 비대칭적으로 적용하여 각 시점이 **현재와 과거 프레임만** 참조하도록 제한한다:

```
일반 Temporal Conv (kernel=3): 프레임 t-1, t, t+1 참조 (미래 정보 누출
Causal Temporal Conv (kernel=3): 프레임 t-2, t-1, t 참조 (미래 정보 차
```

이것은 언어 모델에서 causal attention mask를 사용하는 것과 동일한 원리이며, 비디오를 앞에서부터 순차적으로 생성하거나 스트리밍할 때 필수적이다.

A.7 비디오 생성의 핵심 도전

도전	설명	현재 상태
물리 법칙	물체가 물리적으로 타당하게 움직여야	부분적으로 학습
3D 일관성	카메라가 이동해도 같은 장면	개선 중
장시간 일관성	1분 이상의 서사 일관성	여전히 어려움
계산 비용	이미지 대비 10~100배	최적화 진행 중
학습 데이터	고품질 비디오-텍스트 쌍 부족	자동 캡셔닝 활용

> 💡 **각 도전이 어려운 근본적 이유:**
>
> **물리 법칙**: 비디오 모델은 물리 엔진이 아니라 데이터에서 패턴을 학습하는 통계 모델이다. 중력, 관성, 마찰 같은 물리 법칙을 **명시적으로 인코딩하지 않기 때문에**, 학습 데이터에 충분히 포함되지 않은 물리 현상(예: 무중력 환경, 비뉴턴 유체)은 정확히 재현하지 못한다. 현재 접근법은 (a) 더 많은 물리 시뮬레이션 데이터로 학습하거나, (b) 물리 시뮬레이터와 결합하는 하이브리드 방식이 연구되고 있다.
>
> **3D 일관성**: 비디오 모델은 2D 프레임의 시퀀스를 생성하지, 3D 장면을 구성하는 것이 아니다. 카메라가 물체를 360도 돌면, 앞면과 뒷면이 **하나의 3D 물체에서 나온 것**이라는 보장이 없다. 이것은 모델이 암묵적으로 3D 구조를 이해해야 해결되는 문제이며, Part B에서 다루는 3D 생성과 깊은 관련이 있다.
>
> **장시간 일관성**: Transformer의 context window가 제한되어 있으므로, 비디오가 길어지면 초반부의 정보가 점차 희석된다. 1분 비디오(30fps = 1,800프레임)에서 프

> 레임 1의 정보가 프레임 1,800까지 유지되려면, 극도로 긴 시퀀스를 처리하는 메커니즘이 필요하다. 현재는 hierarchical generation(먼저 저해상도/저프레임 생성 후 점진적 업스케일)이나 autoregressive chunk 방식이 사용된다.

Part B: 3D 생성 (3D Generation)

B.1 2D Diffusion으로 3D를 만드는 방법

> 💡 **왜 이런 접근이 필요한가?** 3D 생성의 가장 직관적인 방법은 "대량의 3D 데이터를 모아서 3D Diffusion 모델을 학습하는 것"이다. 하지만 이것은 현실적으로 불가능하다:
>
> - **이미지-텍스트 쌍**: LAION-5B 등으로 **50억 개** 이상 존재 (인터넷에서 자동 수집 가능)
> - **3D 모델-텍스트 쌍**: Objaverse 등으로 약 **80만 개** 수준 (전문 아티스트가 수작업으로 제작)
> - **격차**: 약 **6,000배** — 이 격차를 데이터 수집으로 메우는 것은 비현실적
>
> 이 상황에서 DreamFusion 팀의 핵심 통찰은: "3D 데이터가 없어도, **2D 이미지의 지식을 빌려올 수 있다**"는 것이다. 2D Diffusion 모델은 이미 "사과는 빨갛고 둥글며, 위에서 보면 원형이고 옆에서 보면 약간 타원형"이라는 다시점(multi-view) 지식을 암묵적으로 학습했다. SDS는 이 암묵적 지식을 추출하여 3D를 구성하는 방법이다.

대규모 3D 데이터셋은 부족하지만, 2D 이미지 데이터는 풍부하다. 핵심 아이디어:

> "2D Diffusion 모델이 학습한 '이미지가 어떻게 보여야 하는가'에 대한 지식을 3D 생성에 활용"

⚠️ **3D 데이터 부족의 현실**: 이미지-텍스트 쌍은 수십억 개가 존재하지만 (LAION-5B 등), 고품질 3D 모델-텍스트 쌍은 수만 개 수준이다. SDS는 이 데이터 격차를 2D 모델의 지식 전이(knowledge transfer)로 극복한 획기적인 아이디어이다.

NeRF (Neural Radiance Field) — 3D 표현의 기초:

SDS를 이해하려면 먼저 NeRF가 무엇인지 알아야 한다. NeRF는 3D 장면을 **연속 함수**로 표현하는 방법이다:

```
NeRF = MLP 신경망 f_θ
입력: 3D 좌표 (x, y, z) + 관측 방향 (θ, ϕ)
출력: 색상 (r, g, b) + 밀도 (σ)

의미: "3D 공간의 점 (x,y,z)를 방향 (θ,ϕ)에서 바라보면
       색상 (r,g,b)이고, 그 점의 물질 밀도는 σ이다"
```

NeRF로 이미지를 렌더링하는 과정 (Volume Rendering):

```
Step 1: 카메라에서 각 픽셀을 향해 광선(ray)을 쏜다
Step 2: 광선 위의 N개 점 (x_1, x_2, ..., x_N)을 샘플링
Step 3: 각 점에서 MLP를 평가: (color_i, σ_i) = f_θ(x_i, direction)
Step 4: Volume rendering으로 색상 합성:
  C(ray) = Σ_i T_i · (1 - exp(-σ_i · δ_i)) · color_i
  여기서 T_i = exp(-Σ_{j<i} σ_j · δ_j)  (투과율: 앞의 물질에 의한 차폐
  δ_i = 인접 샘플 간 거리
```

> 💡 **왜 NeRF가 SDS에 적합한가?** NeRF의 핵심 장점은 **미분 가능한 렌더링 (differentiable rendering)**이다. 렌더링 과정의 모든 연산(MLP 평가, volume rendering 합성)이 미분 가능하므로, 렌더링된 이미지의 손실을 MLP 파라미터 θ까지 역전파할 수 있다: $\frac{\partial \text{image}}{\partial \theta}$가 존재한다. 이것이 SDS가 "렌더링 이미지를 개선하라"는 신호를 받아 3D 표현을 업데이트할 수 있는 이유다. 만약 렌더링이 미분 불가능했다면 (예: 전통적인 래스터화), SDS는 작동할 수 없다.

B.2 Score Distillation Sampling (SDS)

📄 *Poole et al., "DreamFusion: Text-to-3D using 2D Diffusion", ICLR 2023*

> 💡 **왜 SDS가 핵심 돌파구(breakthrough)인가?** SDS 이전에는 text-to-3D가 사실상 불가능했다. 3D 데이터가 너무 적어 3D Diffusion을 직접 학습할 수 없었고, CLIP만으로 3D를 최적화하면 품질이 매우 낮았다. SDS의 핵심 통찰은: "2D Diffusion 모델을 3D 최적화의 **비평가(critic)**로 사용하자"는 것이다. 렌더링 이미지를 Diffusion 모델에게 보여주고, "이것이 자연스러운 이미지인가?"를 묻고, 그 피드백으로 3D 표현을 개선한다. 이것은 GAN에서 판별자(discriminator)가 생성자를 학습시키는 것과 개념적으로 유사하지만, 사전학습된 Diffusion 모델을 판별자로 **재활용**한다는 점이 혁신적이다.

NeRF(Neural Radiance Field)로 3D 장면을 표현하고, 2D Diffusion의 스코어로 최적화:

```
3D 표현 θ (NeRF)
    ↓
[임의 각도에서 렌더링]
    ↓
렌더링 이미지 x = render(θ, camera)
    ↓
[사전학습된 2D Diffusion 모델]
    ↓
"이 이미지가 텍스트와 얼마나 일치하는가?"의 기울기
    ↓
3D 표현 θ 업데이트
```

SDS Loss 유도 과정 (Step-by-Step Derivation):

SDS의 수학적 유도를 단계별로 따라가보자:

Step 1: 목표 설정

우리의 목표는 3D 파라미터 θ를 최적화하여, **어떤 각도에서 렌더링하든** 텍스트 y에 부합하는 자연스러운 이미지가 나오게 하는 것이다.

$$\theta^* = \arg\min_\theta \mathbb{E}_c \left[\mathcal{L}(g(\theta, c), y)\right]$$

여기서 $g(\theta, c)$는 카메라 c에서의 렌더링 이미지다. 문제는: 손실 함수 $\mathcal{L}$을 어떻게 정의할 것인가?

Step 2: Diffusion 모델을 "자연스러움 판단기"로 사용

사전학습된 Diffusion 모델 ϵ_ϕ는 자연스러운 이미지의 분포 $p_{\text{data}}(x)$를 학습했다. Diffusion 모델의 학습 목표는 score function $\nabla_x \log p_t(x)$를 근사하는 것이었다 (강의 7 참조). 따라서:

$$\epsilon_\phi(x_t, t, y) \approx -\sigma_t \nabla_{x_t} \log p_t(x_t|y)$$

즉, Diffusion 모델의 노이즈 예측은 **조건부 스코어 함수의 스케일된 버전**이다.

Step 3: SDS 기울기 유도

렌더링 이미지 $x = g(\theta, c)$가 Diffusion 모델의 학습 분포에 속하도록 하려면, x를 노이즈 레벨 t에서 perturb한 x_t의 스코어를 따라가면 된다:

$$\nabla_\theta \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t,\epsilon,c}\left[w(t)\underbrace{(\epsilon_\phi(x_t, t, y) - \epsilon)}_{\text{update direction}} \underbrace{\frac{\partial x}{\partial \theta}}_{\text{chain rule}}\right]$$

Step 4: 각 항의 의미

항	의미	역할
$\epsilon_\phi(x_t, t, y)$	Diffusion 모델이 예측한 노이즈	"이 이미지를 자연스럽게 만들려면 이 방향으로 수정해야 한다"
ϵ	실제로 추가한 노이즈	우리가 추가한 것이므로 이미 알고 있는 것
$\epsilon_\phi - \epsilon$	두 노이즈의 차이	렌더링 이미지가 자연스러운 분포에서 **얼마나, 어떤 방향으로** 벗어나 있는지
$w(t)$	시간 스텝별 가중치	보통 $w(t) = \sigma_t^2$. 노이즈가 큰 t에서는 큰 스텝, 작은 t에서는 세밀한 조정
$\partial x/\partial \theta$	렌더링의 3D 파라미터 의존성	Chain rule: 이미지 공간의 기울기를 3D 파라미터 공간으로 전달

> **핵심 직관 (비유)**: 미술 선생님(Diffusion 모델)에게 학생(NeRF)의 그림(렌더링)을 보여준다. 선생님은 "여기가 자연스럽지 않다"고 피드백($\epsilon_\phi - \epsilon$)을 준다. 학생은 이 피드백을 받아 자신의 3D 표현을 수정한다. 이것을 수만 번 반복하면, 어느 각도에서 그려도 선생님이 만족하는 3D 모델이 완성된다.

> **직관**: 렌더링 이미지에 노이즈를 추가하고, Diffusion 모델에게 "이 이미지를 어떻게 개선하면 좋겠는가"를 묻는다. 그 답(기울기)을 3D 표현에 전파.

⚠️ **SDS의 수학적 의미**: SDS 기울기 $(\epsilon_\phi(x_t, t, y) - \epsilon)$는 사전학습된 Diffusion 모델이 예측한 노이즈와 실제 추가한 노이즈의 차이이다. 이 차이가 0이 되면 렌더링 이미지가 Diffusion 모델의 학습 분포(즉, 자연스러운 이미지 분포)에 속한다는 뜻이다. $w(t)$는 시간 스텝에 따른 가중치로, 보통 σ_t^2에 비례한다.

SDS에서 Diffusion 모델의 기울기를 차단하는 이유 (stop-gradient):

코드에서 `torch.no_grad()`로 Diffusion 모델의 기울기를 차단하는 것은 단순한 구현 트릭이 아니라, SDS의 **수학적 정의의 일부**이다. SDS는 Diffusion 모델 ϵ_ϕ를 상수로 취급하고, 오직 렌더링 함수 $g(\theta, c)$를 통한 3D 파라미터 θ만 최적화한다. Diffusion 모델까지 미분하면 완전히 다른 기울기가 되어, 3D 표현이 수렴하지 않는다. 이것은 GAN 학습에서 판별자를 고정하고 생성자만 업데이트하는 것과 같은 원리다.

B.3 PyTorch 구현: Simplified SDS Loss

```python
import torch
import torch.nn as nn

class SimplifiedSDSLoss(nn.Module):
    """Score Distillation Sampling 손실의 단순화된 구현.

    핵심 과정:
    1. 3D 표현에서 렌더링한 이미지에 노이즈를 추가
    2. 사전학습된 Diffusion 모델로 노이즈를 예측
    3. 예측 노이즈와 실제 노이즈의 차이를 기울기로 사용
    """
    def __init__(self, pretrained_diffusion_model, num_timesteps: i
        super().__init__()
        # 사전학습된 2D Diffusion 모델 (고정, 학습하지 않음)
        self.diffusion_model = pretrained_diffusion_model
        for param in self.diffusion_model.parameters():
            param.requires_grad_(False)

        self.num_timesteps = num_timesteps

        # DDPM 스케줄 (간소화)
        betas = torch.linspace(1e-4, 0.02, num_timesteps)
        alphas = 1.0 - betas
        self.register_buffer("alpha_bar", torch.cumprod(alphas, dim

    def forward(
        self,
        rendered_image: torch.Tensor,   # [B, C, H, W] 렌더링된 이미
        text_embedding: torch.Tensor,    # [B, L, D] 텍스트 조건 임베
        guidance_scale: float = 100.0,   # CFG 스케일 (SDS에서는 높게
    ) -> torch.Tensor:
        """
        SDS 기울기를 계산하여 rendered_image에 역전파 가능한 손실을 반환

        실제로는 ▽_θ L_SDS를 직접 계산하지만,
        PyTorch autograd를 활용하기 위해 surrogate loss를 구성한다.
        """
        B = rendered_image.shape[0]
        device = rendered_image.device

        # 1. 랜덤 시간 스텝 샘플링 (너무 작거나 큰 t는 제외)
        t = torch.randint(20, 980, (B,), device=device)

        # 2. 노이즈 추가 (forward diffusion)
        alpha_bar_t = self.alpha_bar[t].view(B, 1, 1, 1)   # [B, 1,
        epsilon = torch.randn_like(rendered_image)           # 실제

        # x_t = √ᾱ_t * x + √(1-ᾱ_t) * ε
        x_noisy = torch.sqrt(alpha_bar_t) * rendered_image + \
                  torch.sqrt(1 - alpha_bar_t) * epsilon

        # 3. Diffusion 모델의 노이즈 예측 (기울기 차단!)
```

```
        with torch.no_grad():
            # CFG 적용: conditional + unconditional 예측
            noise_pred_cond = self.diffusion_model(x_noisy, t, text_
            noise_pred_uncond = self.diffusion_model(x_noisy, t, Noı

            # Classifier-Free Guidance
            noise_pred = noise_pred_uncond + guidance_scale * (noise

        # 4. SDS 기울기 계산: w(t) * (ε_pred - ε)
        # w(t) = σ_t^2 로 근사 (실제로는 다양한 가중치 스케줄 사용)
        w = (1 - alpha_bar_t)
        grad = w * (noise_pred - epsilon)

        # 5. Surrogate loss: rendered_image의 기울기가 grad가 되도록
        # ∂L/∂x = grad 이 되도록: L = grad·x (grad는 상수 취급)
        loss = (grad.detach() * rendered_image).sum() / B

        return loss


# --- 사용 예시 (DreamFusion 학습 루프 간소화) ---
def dreamfusion_training_step(
    nerf_model,              # 학습할 3D 표현 (NeRF)
    sds_loss_fn,             # SimplifiedSDSLoss 인스턴스
    text_embedding,          # 텍스트 임베딩 (고정)
    optimizer,               # NeRF 파라미터 옵티마이저
):
    """DreamFusion 학습의 한 스텝."""
    optimizer.zero_grad()

    # 1. 랜덤 카메라 각도 선택
    camera = sample_random_camera()  # 구현은 별도

    # 2. NeRF에서 이미지 렌더링 (미분 가능!)
    rendered_image = nerf_model.render(camera)  # [1, 3, H, W]

    # 3. SDS 손실 계산
    loss = sds_loss_fn(rendered_image, text_embedding)

    # 4. 역전파 → NeRF 파라미터 업데이트
    loss.backward()
    optimizer.step()

    return loss.item()
```

B.4 3D 생성 파이프라인 (DreamFusion)

> 💡 **DreamFusion의 학습 과정을 직관적으로 이해하기** DreamFusion은 **조각가가 점토를 빚는 과정**과 유사하다. 조각가(optimizer)는 처음에 무작위 형태의 점토(랜덤 NeRF)에서 시작한다. 매 반복마다 조각가는 (a) 랜덤한 각도에서 점토를 바라보

> 고(렌더링), (b) 미술 비평가(Diffusion 모델)에게 "이게 공작새처럼 보이나요?"라고 묻고, (c) 비평가의 피드백에 따라 점토를 조금씩 수정한다. 10,000번 반복하면, 어느 각도에서 봐도 "서핑보드 위의 공작새"로 보이는 3D 조각이 완성된다.

```
입력: "a DSLR photo of a peacock on a surfboard"

1. NeRF θ 랜덤 초기화
2. 반복 (10000 iterations):
   a. 랜덤 카메라 각도 c 선택
   b. x = render(θ, c)
   c. t ~ Uniform, ε ~ N(0,I)
   d. x_t = √ᾱ_t · x + √(1-ᾱ_t) · ε
   e. ε^= pretrained_diffusion(x_t, t, "a DSLR photo of a peacock o
   f. ▽_θ = (ε^- ε) · ∂x/∂θ
   g. θ ← θ - η · ▽_θ
3. 결과: 어떤 각도에서도 텍스트에 부합하는 3D 장면
```

각 단계의 세부 설명:

- **Step a (카메라 샘플링)**: 구면 좌표계에서 균일하게 샘플링한다. 상반구(위에서 보는 각도)와 하반구를 모두 포함해야 물체의 아래쪽도 제대로 학습된다. 실제로는 elevation 각도에 따라 텍스트 프롬프트를 변경하기도 한다 (예: 위에서 보면 "overhead view of...", 옆에서 보면 "side view of..." 추가).
- **Step c (시간 스텝 선택)**: 큰 t (강한 노이즈)에서는 전체적인 구조(형태, 색상)가 업데이트되고, 작은 t (약한 노이즈)에서는 세부 디테일(텍스처, 엣지)이 정교해진다. 학습 초기에는 큰 t를 주로 사용하고, 후기에는 작은 t를 사용하는 스케줄링(annealing)이 효과적이다.
- **Step f (기울기 계산)**: $\partial x / \partial \theta$는 NeRF의 volume rendering을 통한 기울기로, PyTorch의 autograd가 자동으로 계산한다. 이 기울기는 "렌더링 이미지의 이 픽셀이 밝아져야 한다면, NeRF의 어떤 3D 점의 색상/밀도를 변경해야 하는가?"를 알려준다.

B.5 후속 모델

모델	년도	핵심 개선
DreamFusion	2022	SDS + NeRF (최초 Text-to-3D)
Magic3D	2022	2단계: Coarse (NeRF) → Fine (Mesh)
ProlificDreamer	2023	VSD (Variational SDS) → 품질 향상
Zero-1-to-3	2023	단일 이미지 → 3D
Instant3D	2023	빠른 3D 생성 (수 초)

Janus Problem (야누스 문제) — SDS의 핵심 한계:

DreamFusion으로 사람 얼굴이나 동물을 생성하면, 종종 **앞뒤 양면에 모두 얼굴이 생기는** 기이한 결과가 나온다. 이것을 로마 신화의 양면 신 야누스(Janus)의 이름을 따 "Janus problem"이라 부른다.

```
정상적인 3D 모델:          Janus Problem:
  앞면: 얼굴 😊              앞면: 얼굴 😊
  뒷면: 뒤통수               뒷면: 얼굴 😊 (또 얼굴!)
  옆면: 옆모습               옆면: 기이한 전환
```

> 💡 **왜 Janus Problem이 발생하는가?** 근본 원인은 2D Diffusion 모델이 **각 뷰를 독립적으로 평가**한다는 데 있다. 정면에서 렌더링한 이미지를 Diffusion 모델에 보여주면 "얼굴이 보여야 한다"고 피드백하고, 뒷면에서 렌더링한 이미지를 보여줘도 "얼굴이 보여야 한다"고 피드백한다 — 왜냐하면 2D Diffusion 모델은 "고양이 사진"이라고 하면 대부분 정면 얼굴을 기대하기 때문이다. Diffusion 모델은 앞면과 뒷면이 **같은 3D 물체의 서로 다른 면**이라는 것을 알지 못한다. 각 뷰를 독립적으로 "가장 그럴듯한 이미지"로 만들려다 보니, 모든 면에 가장 빈도 높은 패턴(얼굴)이 나타나는 것이다.
>
> 이것은 SDS의 구조적 한계이며, 해결 방법으로는:
>
> - **View-dependent prompting**: 카메라 각도에 따라 "front view", "back view", "side view" 등의 접두사를 프롬프트에 추가
> - **Multi-View Diffusion**: 여러 시점의 이미지를 동시에 생성하는 모델 사용 (B.6 참조)
> - **3D-aware Diffusion**: 3D 구조를 인식하는 Diffusion 모델 학습

📄 *Lin et al., "Magic3D: High-Resolution Text-to-3D Content Creation", CVPR 2023* 📄 *Wang et al., "ProlificDreamer: High-Fidelity and Diverse Text-to-3D Generation with Variational Score Distillation", NeurIPS 2023* 📄 *Liu et al., "Zero-1-to-3: Zero-shot One Image to 3D Object", ICCV 2023*

> 💡 **ProlificDreamer의 VSD는 SDS를 어떻게 개선하는가?** SDS의 또 다른 문제는 **과포화(over-saturation)**와 **다양성 부족**이다. SDS는 CFG scale을 매우 높게 (100 이상) 설정해야 하는데, 이것은 색상이 과하게 채도가 높아지고, 항상 비슷한 결과만 나오게 한다. ProlificDreamer의 VSD (Variational Score Distillation)는 SDS의 단일 점 추정을 **분포 추정**으로 확장하여, 더 자연스러운 색상과 높은 다양성을 달성한다. 수학적으로 VSD는 SDS 기울기에서 추가적인 score term을 빼서, 과도한 업데이트를 억제한다.

B.6 Multi-View Diffusion

> 💡 **왜 Multi-View Diffusion이 SDS의 대안으로 등장했는가?** SDS는 획기적이었지만 실용적 한계가 분명하다: (1) **속도** — 10,000+ iteration의 최적화가 필요해서 한 객체 생성에 1~2시간 소요, (2) **Janus problem** — 2D 모델이 각 뷰를 독립적으로 판단하여 다시점 일관성이 깨짐, (3) **과포화 및 낮은 다양성** — 높은 CFG scale로

> 인한 부작용. Multi-View Diffusion은 이 세 문제를 동시에 해결하는 패러다임 전환이다. "3D를 최적화하는 대신, 다시점 이미지를 한번에 생성하자"는 발상이다.

SDS의 한계(느리고 모드 붕괴 가능)를 극복하는 최신 접근:

1. 다시점(multi-view) 이미지를 직접 생성하는 Diffusion 모델 학습
2. 생성된 다시점 이미지로 3D 재구성

```
텍스트 → [Multi-View Diffusion] → [정면, 좌측, 우측, 후면] 이미지
                                              ↓
                                  [3D Reconstruction] → 3D Mesh
```

Multi-View Diffusion의 핵심 구조:

Multi-View Diffusion은 일반 이미지 Diffusion 모델을 확장하여, **여러 시점의 이미지를 동시에 생성**한다. 핵심은 서로 다른 뷰들 사이에 attention을 추가하는 것이다:

```
[뷰 1 토큰] ←— Cross-View Attention —→ [뷰 2 토큰]
     ↕                                      ↕
[뷰 3 토큰] ←— Cross-View Attention —→ [뷰 4 토큰]
```

이 cross-view attention 덕분에, "정면에 눈이 있으면 뒷면에는 뒤통수가 있어야 한다"는 다시점 일관성이 자연스럽게 학습된다. 이것은 비디오 생성의 temporal attention과 개념적으로 동일하다 — 비디오에서 "시간 축 일관성"이 필요했듯, 여기서는 "시점 축 일관성"이 필요하다.

⚠️ **SDS vs Multi-View Diffusion**: SDS는 최적화 기반으로 10,000+ iteration이 필요하여 느리지만, Multi-View Diffusion은 피드포워드(feed-forward) 방식으로 수 초 만에 3D를 생성할 수 있다. 최근에는 두 방법을 결합하여 Multi-View 생성 + SDS 정제(refinement)를 수행하는 하이브리드 접근도 연구되고 있다.

비교 정리:

측면	SDS (DreamFusion)	Multi-View Diffusion
속도	1~2시간 (최적화 기반)	수 초 (피드포워드)
다시점 일관성	약함 (Janus problem)	강함 (cross-view attention)
디테일	높음 (반복 정제)	중간 (단일 패스)
학습 데이터	2D 이미지만 필요	다시점 3D 데이터 필요
최신 트렌드	Multi-View + SDS 정제 하이브리드 결합	

Part C: Physical AI — Diffusion for Robotics

C.1 왜 Diffusion이 로봇에 필요한가?

> 💡 **왜 기존 로봇 학습 방법이 실패하는가?** 로봇 학습의 가장 일반적인 접근인 **Behavior Cloning**은 "전문가가 보여준 행동을 모방한다"는 단순한 아이디어다. 전문가의 시연 데이터 $\{(o_i, a_i)\}$ (관측-행동 쌍)을 수집하고, 신경망 $\pi_\theta(o) \approx a$를 학습한다. MSE 손실 $\mathcal{L} = \|a - \pi_\theta(o)\|^2$를 최소화하면 된다.
>
> 이것은 **행동이 유일(unimodal)할 때**는 잘 작동한다. 하지만 현실의 로봇 태스크에서는 **같은 상태에서 여러 개의 올바른 행동**이 존재하는 경우가 매우 많다. 장애물을 왼쪽으로 돌아갈 수도, 오른쪽으로 돌아갈 수도 있다. 컵을 손잡이를 잡을 수도, 몸통을 잡을 수도 있다. 이때 MSE 손실은 수학적으로 **조건부 기댓값** $\mathbb{E}[a|o]$로 수렴하는데, 이것은 두 유효한 행동의 평균이 되어 **어느 쪽도 아닌 무효한 행동**이 된다.

전통적 로봇 학습:

- **Behavior Cloning**: 시연 데이터에서 상태→행동 매핑을 학습
- **문제**: 같은 상태에서 여러 유효한 행동이 있을 때, 평균을 내면 **무효한 행동**이 됨

```
예: 테이블 위의 컵을 잡는 두 가지 방법
  방법 A: 왼쪽에서 잡기
  방법 B: 오른쪽에서 잡기
  평균: 가운데서 잡기 (실패!)
```

수학적으로 왜 평균화가 재앙인가:

```
상황: 장애물 앞에서 왼쪽(a=-1) 또는 오른쪽(a=+1)으로 가야 함
시연 데이터: 50%는 a=-1, 50%는 a=+1

MSE 회귀 모델의 예측:
  argmin_a E[(a - a_demo)²]
  = argmin_a [0.5(a-(-1))² + 0.5(a-(+1))²]
  = argmin_a [a² + 1]    (미분하면 2a = 0)
  = a* = 0               ← 정면 충돌!

Diffusion 모델의 예측:
  p(a|o)를 학습 → bimodal 분포 (a=-1 근처 피크 + a=+1 근처 피크)
  샘플링 → a=-1 또는 a=+1 중 하나를 선택 ← 안전!
```

Diffusion은 **다중 모드(multimodal) 분포**를 표현할 수 있다 → 유효한 행동 중 하나를 샘플링.

⚠️ **Multimodal Distribution 문제의 본질**: MSE 손실을 사용하는 회귀(regression) 모델은 $\mathbb{E}[a|o]$를 예측한다. 행동 분포가 bimodal이면 이 기댓값은 두 모드 사이의 빈 영

역에 위치하게 된다. Diffusion은 전체 분포 $p(a|o)$를 학습하므로 이 문제를 본질적으로 회피한다. 이것은 GAN도 할 수 있지만, Diffusion이 GAN보다 학습이 안정적이라는 장점이 있다.

> 💡 **왜 GAN이 아닌 Diffusion인가?** GAN도 multimodal 분포를 표현할 수 있지만, 로봇 학습에서 Diffusion이 선호되는 이유는: (1) **학습 안정성** — GAN은 mode collapse, training instability 등의 문제가 있어 하이퍼파라미터 튜닝이 어렵다. 로봇 학습에서는 "학습이 안정적으로 수렴한다"는 보장이 안전에 직결된다. (2) **조건부 생성의 용이함** — 관측을 조건으로 넣어 행동을 생성하는 것이 Diffusion에서는 자연스럽다 (cross-attention 또는 concatenation). (3) **행동 시퀀스 생성** — 시계열 데이터를 다루는 데 Diffusion이 더 적합하다는 실험적 증거가 많다.

C.2 Diffusion Policy

📄 *Chi et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion", RSS 2023*

> 💡 **Diffusion Policy의 핵심 아이디어** 이미지 생성에서 Diffusion은 "노이즈 → 이미지"를 생성한다. Diffusion Policy는 정확히 같은 원리를 "노이즈 → 행동 시퀀스" 생성에 적용한다. 이미지 Diffusion에서 텍스트가 조건이었듯, 여기서는 **로봇의 현재 관측(카메라 이미지 + 관절 상태)**이 조건이다. Diffusion 모델의 수학은 완전히 동일하고, 단지 생성하는 **대상의 의미**만 바뀐다.

로봇의 행동을 **Diffusion으로 생성**:

```
관측 o_t (카메라 + 로봇 상태)
    ↓
[관측 인코더 (CNN/ViT)]
    ↓
조건 벡터 c
    ↓
a_T ~ N(0, I)    ← 행동 공간의 노이즈
    ↓
[DDPM Reverse Process, K steps]
    ↓
행동 시퀀스 a₀ = [a_t, a_{t+1}, ..., a_{t+H}]  ← H 스텝의 행동 계획
    ↓
로봇 실행 (첫 몇 스텝만 실행 후 재계획)
```

> 💡 **왜 단일 행동이 아닌 행동 시퀀스(action sequence)를 예측하는가?** 단일 행동 a_t만 예측하면 세 가지 문제가 있다:
>
> (1) **시간적 불일관성**: 매 timestep마다 독립적으로 행동을 샘플링하면, 연속된 행동들이 서로 모순될 수 있다. 한 순간은 왼쪽으로, 다음 순간은 오른쪽으로 가는 "떨림(jittering)" 현상이 발생한다.

> (2) **계획 불가**: 단일 행동은 "지금 당장 뭘 할까?"만 고려한다. 하지만 "컵을 집으려면 먼저 손을 벌리고, 접근하고, 닫아야 한다"와 같은 순차적 계획이 필요하다. 시퀀스 예측은 이런 multi-step planning을 자연스럽게 인코딩한다.
>
> (3) **실행 효율성**: 매 프레임마다 Diffusion의 전체 디노이징 과정(예: 100스텝)을 수행하면 너무 느리다. 대신 H스텝의 행동을 한번에 생성하고, 첫 몇 스텝만 실행한 뒤 재계획하면 **계산을 H배 절약**할 수 있다.
>
> 이 "시퀀스 생성 + 일부만 실행 + 재계획" 패턴을 **Receding Horizon Control** (또는 Model Predictive Control, MPC)이라 하며, 로봇 제어의 표준적인 방법론이다.

행동 시퀀스 예측: 단일 행동이 아닌 미래 H 스텝의 행동을 한꺼번에 생성.

DDPM과 동일한 수식, 단 이미지 대신 **행동 시퀀스**에 적용:

$$\mathcal{L} = \mathbb{E}_{t,a_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(a_t, t, o)\|^2\right]$$

여기서:

- a_0: 시연 데이터의 행동 시퀀스 ($\mathbb{R}^{H\times d_{\text{action}}}$)
- o: 관측 (이미지 + 로봇 상태)
- $a_t = \sqrt{\bar{\alpha}_t}a_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$

이미지 Diffusion과 Diffusion Policy의 대응 관계:

이미지 Diffusion	Diffusion Policy	의미
이미지 $x \in \mathbb{R}^{H\times W\times 3}$	행동 시퀀스 $a \in \mathbb{R}^{H_a\times d_a}$	생성 대상
텍스트 프롬프트 y	관측 o (카메라 + 상태)	조건
U-Net / DiT	1D U-Net / Transformer	노이즈 예측 네트워크
1000 디노이징 스텝	10~100 디노이징 스텝	추론 스텝 수
이미지 품질	행동의 실행 가능성, 부드러움	평가 기준

행동 공간의 차원이 이미지(수만~수십만 픽셀)보다 훨씬 작기 때문에($H_a \times d_a$는 보통 16x7 = 112 정도), Diffusion 스텝 수를 10~100으로 대폭 줄여도 충분히 좋은 결과를 얻을 수 있다. 이것은 로봇의 실시간 제어에 매우 중요한 장점이다.

C.3 PyTorch 구현: Diffusion Policy 학습 및 추론

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class DiffusionPolicyNetwork(nn.Module):
    """Diffusion Policy의 노이즈 예측 네트워크.

    조건(관측)이 주어졌을 때, 노이즈가 추가된 행동 시퀀스에서
    원래 노이즈를 예측한다. (DDPM의 ε_θ에 해당)

    실제 구현에서는 1D U-Net 또는 Transformer를 사용하지만,
    여기서는 간소화된 MLP 기반 구조를 사용한다.
    """
    def __init__(
        self,
        action_dim: int = 7,          # 로봇 행동 차원 (예: 7-DoF 로
        action_horizon: int = 16,      # 예측할 행동 시퀀스 길이 H
        obs_dim: int = 512,            # 관측 인코더 출력 차원
        hidden_dim: int = 256,         # 히든 레이어 차원
        time_embed_dim: int = 128,     # 시간 임베딩 차원
    ):
        super().__init__()
        self.action_dim = action_dim
        self.action_horizon = action_horizon
        total_action_dim = action_dim * action_horizon  # 전체 행동

        # 시간 스텝 임베딩 (sinusoidal, DDPM과 동일)
        self.time_embed = nn.Sequential(
            SinusoidalPosEmb(time_embed_dim),
            nn.Linear(time_embed_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
        )

        # 관측 인코더 (사전학습된 CNN/ViT 특징을 사용한다고 가정)
        self.obs_encoder = nn.Sequential(
            nn.Linear(obs_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
        )

        # 노이즈 예측 네트워크 (행동 + 시간 + 관측 → 노이즈 예측)
        self.noise_pred_net = nn.Sequential(
            nn.Linear(total_action_dim + hidden_dim + hidden_dim, h
            nn.SiLU(),
            nn.Linear(hidden_dim * 4, hidden_dim * 4),
            nn.SiLU(),
            nn.Linear(hidden_dim * 4, hidden_dim * 2),
            nn.SiLU(),
            nn.Linear(hidden_dim * 2, total_action_dim),  # 노이즈와
        )
```

```
        ,

    def forward(
        self,
        noisy_action: torch.Tensor,    # [B, H, action_dim] 노이즈가
        timestep: torch.Tensor,         # [B] 현재 디퓨전 시간 스텝
        obs_feature: torch.Tensor,      # [B, obs_dim] 관측 인코더 출
    ) -> torch.Tensor:
        """노이즈 예측: ε_θ(a_t, t, o)"""
        B = noisy_action.shape[0]

        # 행동 시퀀스를 플랫하게: [B, H, action_dim] → [B, H * action_
        action_flat = noisy_action.view(B, -1)

        # 시간 임베딩
        t_embed = self.time_embed(timestep)        # [B, hidden_dim_

        # 관측 인코딩
        obs_embed = self.obs_encoder(obs_feature)  # [B, hidden_dim_

        # 결합 후 노이즈 예측
        combined = torch.cat([action_flat, t_embed, obs_embed], dim=
        noise_pred = self.noise_pred_net(combined)

        # 원래 형태로: [B, H * action_dim] → [B, H, action_dim]
        return noise_pred.view(B, self.action_horizon, self.action_(


class SinusoidalPosEmb(nn.Module):
    """DDPM과 동일한 Sinusoidal 시간 임베딩."""
    def __init__(self, dim: int):
        super().__init__()
        self.dim = dim

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        device = t.device
        half = self.dim // 2
        emb = torch.log(torch.tensor(10000.0)) / (half - 1)
        emb = torch.exp(torch.arange(half, device=device) * -emb)
        emb = t[:, None].float() * emb[None, :]
        return torch.cat([emb.sin(), emb.cos()], dim=-1)


class DiffusionPolicyTrainer:
    """Diffusion Policy 학습 및 추론 루프.

    학습: 시연 데이터의 행동 시퀀스에 노이즈를 추가하고, 이를 예측하도록
    추론: 순수 노이즈에서 DDPM 역과정을 통해 행동 시퀀스 생성
    """
    def __init__(
        self,
        policy_net: DiffusionPolicyNetwork,
        num_diffusion_steps: int = 100,    # 로봇은 이미지보다 적은 스
```

```
    num_diffusion_steps: int = 100,   # 보통은 이미지보다 훨씬 ㅈ
    device: str = "cuda",
):
    self.policy_net = policy_net.to(device)
    self.num_steps = num_diffusion_steps
    self.device = device

    # DDPM 노이즈 스케줄
    betas = torch.linspace(1e-4, 0.02, num_diffusion_steps, dev
    alphas = 1.0 - betas
    alpha_bar = torch.cumprod(alphas, dim=0)

    self.betas = betas
    self.alphas = alphas
    self.alpha_bar = alpha_bar

    self.optimizer = torch.optim.Adam(policy_net.parameters(), 

# ==================== 학습 (Training) ====================
def train_step(
    self,
    action_seq: torch.Tensor,     # [B, H, action_dim] 시연 데이
    obs_feature: torch.Tensor,     # [B, obs_dim] 관측 인코더 출
) -> float:
    """DDPM 학습의 한 스텝 (행동 시퀀스 버전).

    L = E_{t,ε} [ || ε - ε_θ(a_t, t, o) ||² ]
    """
    self.policy_net.train()
    B = action_seq.shape[0]

    # 1. 랜덤 시간 스텝 샘플링
    t = torch.randint(0, self.num_steps, (B,), device=self.devi

    # 2. 노이즈 추가 (forward process)
    epsilon = torch.randn_like(action_seq)
    alpha_bar_t = self.alpha_bar[t].view(B, 1, 1)

    # a_t = √ᾱ_t * a_0 + √(1-ᾱ_t) * ε
    noisy_action = torch.sqrt(alpha_bar_t) * action_seq + \
                   torch.sqrt(1 - alpha_bar_t) * epsilon

    # 3. 노이즈 예측
    noise_pred = self.policy_net(noisy_action, t, obs_feature)

    # 4. MSE 손실
    loss = F.mse_loss(noise_pred, epsilon)

    # 5. 역전파 및 업데이트
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
```

```python
        return loss.item()

    # ==================== 추론 (Inference) ====================
    @torch.no_grad()
    def predict_action(
        self,
        obs_feature: torch.Tensor,      # [1, obs_dim] 현재 관측
        num_exec_steps: int = 8,        # 실제 실행할 스텝 수 (< H)
    ) -> torch.Tensor:
        """DDPM 역과정으로 행동 시퀀스를 생성하고, 앞부분만 실행.

        a_T ~ N(0, I) → a_{T-1} → ... → a_0 (깨끗한 행동 시퀀스)
        """
        self.policy_net.eval()
        H = self.policy_net.action_horizon
        action_dim = self.policy_net.action_dim

        # 1. 순수 노이즈에서 시작 (행동 공간)
        a_t = torch.randn(1, H, action_dim, device=self.device)

        # 2. DDPM 역과정 (T → 0)
        for step in reversed(range(self.num_steps)):
            t_tensor = torch.tensor([step], device=self.device)

            # 노이즈 예측
            noise_pred = self.policy_net(a_t, t_tensor, obs_feature)

            # DDPM 업데이트 규칙
            alpha_t = self.alphas[step]
            alpha_bar_t = self.alpha_bar[step]
            beta_t = self.betas[step]

            # 평균 계산: μ = (1/√α_t) * (a_t - (β_t/√(1-ᾱ_t)) * ε_pr
            mean = (1 / torch.sqrt(alpha_t)) * (
                a_t - (beta_t / torch.sqrt(1 - alpha_bar_t)) * nois
            )

            # 노이즈 추가 (마지막 스텝 제외)
            if step > 0:
                noise = torch.randn_like(a_t)
                sigma = torch.sqrt(beta_t)
                a_t = mean + sigma * noise
            else:
                a_t = mean

        # 3. 생성된 행동 시퀀스에서 앞부분만 반환 (receding horizon)
        # 나머지는 다음 관측에서 재계획
        return a_t[:, :num_exec_steps, :]  # [1, num_exec_steps, ac

# --- 전체 Diffusion Policy 학습 & 추론 예시 ---
```

```python
if __name__ == "__main__":
    device = "cuda" if torch.cuda.is_available() else "cpu"

    # 모델 생성
    policy_net = DiffusionPolicyNetwork(
        action_dim=7,           # 7-DoF 로봇팔
        action_horizon=16,      # 16 스텝의 행동 시퀀스 예측
        obs_dim=512,            # ResNet 관측 인코더 출력
    )

    trainer = DiffusionPolicyTrainer(
        policy_net=policy_net,
        num_diffusion_steps=100,
        device=device,
    )

    # === 학습 ===
    print("=== Diffusion Policy 학습 ===")
    for epoch in range(10):
        # 시연 데이터 로드 (실제로는 DataLoader 사용)
        demo_actions = torch.randn(32, 16, 7, device=device)     #
        demo_obs = torch.randn(32, 512, device=device)            #

        loss = trainer.train_step(demo_actions, demo_obs)
        if epoch % 2 == 0:
            print(f"  Epoch {epoch}: loss = {loss:.4f}")

    # === 추론 (로봇 실행) ===
    print("\n=== Diffusion Policy 추론 (로봇 실행) ===")
    current_obs = torch.randn(1, 512, device=device)  # 현재 관측

    # 행동 시퀀스 생성 (100 디노이징 스텝)
    actions = trainer.predict_action(current_obs, num_exec_steps=8)
    print(f"  생성된 행동 시퀀스: {actions.shape}")  # [1, 8, 7]
    print(f"  첫 번째 행동: {actions[0, 0, :].cpu().numpy()}")
```

C.4 Diffusion Policy의 장점

기존 방법	Diffusion Policy
단일 행동 예측 → 다중 모드 평균화	**다중 모드 분포 표현**
결정론적 → 과적합	확률적 → **일반화**
단기 행동 → 근시안적	행동 시퀀스 → **시간적 일관성**

> 💡 **실험적 증거 — Diffusion Policy가 실제로 얼마나 좋은가?** Chi et al. (2023)의 논문에서, Diffusion Policy는 11개의 벤치마크 태스크 중 **10개에서 기존 최고 성능을 초과**했다. 특히 multimodal 행동이 필요한 태스크(예: Push-T — T자 블록을 밀어서 목표 위치에 맞추기)에서 기존 방법 대비 **46.7% 성능 향상**을 보였다. 이것은 multimodal 분포 표현의 중요성을 실증적으로 입증한다.

> 또한 Diffusion Policy는 시연 데이터의 양이 적을 때에도 robust한 성능을 보이는데, 이것은 확률적 모델이 결정론적 모델보다 **과적합에 강하다**는 이론적 예측과 일치한다.

C.5 World Models

> 💡 **왜 World Model이 AI의 다음 프론티어인가?** 현재 대부분의 AI 시스템은 **반응적(reactive)**이다. 관측을 받으면 즉시 행동을 출력한다. 중간에 "만약 이렇게 하면 어떻게 될까?"라는 **시뮬레이션(mental simulation)** 과정이 없다. 인간은 행동하기 전에 머릿속에서 결과를 상상한다 — "이 블록을 밀면 저 블록이 쓰러질 것이다." 이것이 **World Model**이다: 환경의 동역학(dynamics)을 내부적으로 시뮬레이션하는 모델.
>
> World Model이 있으면 **계획(planning)**이 가능해진다:
>
> - 현재 상태에서 가능한 행동 a_1, a_2, a_3을 상상
> - 각 행동의 결과 o'_1, o'_2, o'_3을 World Model로 예측
> - 가장 좋은 결과를 주는 행동 선택
>
> 이것은 체스에서 "수읽기"를 하는 것과 같다. 하지만 체스와 달리, 물리 세계는 연속적이고 시각적이므로, World Model의 예측이 **비디오 예측** 형태를 취하게 된다.

환경을 시뮬레이션하는 비디오 예측 모델:

$$\hat{o}_{t+1} = f_\theta(o_{t-k:t}, a_{t-k:t})$$

과거 k 프레임의 관측과 행동으로 다음 관측을 예측.

```
[과거 관측 + 행동] → [Video Diffusion Model] → [미래 관측 예측]
                                                      ↓
                                                [행동 계획에 활용]
                                                "이 행동을 하면 어떤 결과
```

> 💡 **비디오 예측이 곧 World Modeling인 이유** 다음 프레임을 정확히 예측하려면, 모델은 암묵적으로 다음을 이해해야 한다:
>
> - **물체 영속성(object permanence)**: 물체가 가려져도 존재한다
> - **중력**: 지지대가 없으면 물체가 떨어진다
> - **충돌**: 두 물체가 같은 공간을 차지할 수 없다
> - **인과관계**: "손이 블록을 밀면 블록이 움직인다"
>
> 즉, 비디오 예측 모델은 물리 법칙의 **암묵적 모델(implicit model)**을 학습해야 정확한 예측이 가능하다. 이것이 OpenAI가 Sora를 "world simulator"라고 부른 이유이며, 동시에 논쟁의 대상이 되는 지점이다 — 통계적 패턴 매칭이 진정한 "물리 이해"인가?

World Model의 핵심 구분 — 픽셀 공간 vs 잠재 공간:

```
픽셀 공간 World Model:
  입력: 과거 프레임 [이미지들] + 행동
  출력: 미래 프레임 [이미지들]
  장점: 직관적, 해석 가능
  단점: 고차원, 계산 비용 높음, 세부 디테일에 자원 낭비

잠재 공간 World Model (Yann LeCun의 JEPA 등):
  입력: 과거 프레임의 잠재 표현 + 행동
  출력: 미래 프레임의 잠재 표현
  장점: 저차원, 추상적 계획 가능, 효율적
  단점: 해석 어려움, 잠재 공간의 질에 의존

LeCun은 "세계 모델은 픽셀을 예측할 필요가 없다"고 주장한다.
배경의 나뭇잎 하나하나를 예측하는 것은 불필요하며,
행동 계획에 필요한 추상적 정보만 예측하면 된다는 것이다.
```

모델	기관	년도	핵심
UniSim	Google	2023	범용 시뮬레이터
Genie	Google DeepMind	2024	상호작용 가능한 월드 모델
Cosmos	NVIDIA	2024	로봇/자율주행용 월드 모델

> 💡 **Genie의 혁신 — 행동 없이 World Model 학습하기** 기존 World Model은 (관측, 행동, 다음 관측) 삼중쌍(triple)이 필요했다. 하지만 인터넷의 대부분의 비디오에는 "어떤 행동이 수행되었는가"의 레이블이 없다. Genie는 비디오만으로 World Model을 학습하는 방법을 제안했다: 비디오의 연속 프레임 사이의 변화를 분석하여 **잠재 행동(latent action)**을 자동으로 추론한다. 학습 후에는 사용자가 잠재 행동을 입력하여 생성된 환경과 상호작용할 수 있다. 이것은 수억 개의 레이블 없는 비디오를 World Model 학습에 활용할 수 있는 길을 열었다.

📄 *Yang et al., "Learning Interactive Real-World Simulators (UniSim)", ICLR 2024* 📄 *Bruce et al., "Genie: Generative Interactive Environments", ICML 2024* 📄 *NVIDIA, "Cosmos World Foundation Models", Technical Report, 2024*

C.6 PyTorch 구현: Simplified World Model

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class SimplifiedWorldModel(nn.Module):
    """단순화된 World Model: 과거 관측 + 행동 → 다음 관측 예측.

    실제 모델은 Video Diffusion을 사용하지만,
    여기서는 개념 이해를 위해 조건부 U-Net 기반 Diffusion으로 구현.

    핵심 아이디어:
    - 과거 k프레임의 관측(이미지)과 행동을 조건으로 받아
    - 다음 프레임을 Diffusion으로 생성
    - "이 행동을 하면 어떤 장면이 펼쳐지는가?"를 학습
    """
    def __init__(
        self,
        obs_channels: int = 3,          # 관측 이미지 채널 (RGB)
        action_dim: int = 7,             # 행동 차원
        context_frames: int = 4,         # 과거 관측 프레임 수 k
        hidden_dim: int = 128,           # 히든 차원
        time_embed_dim: int = 64,        # 디퓨전 시간 임베딩 차원
    ):
        super().__init__()
        self.context_frames = context_frames

        # 과거 프레임 인코더: k개의 과거 프레임을 하나의 조건 벡터로 압
        self.context_encoder = nn.Sequential(
            # k프레임을 채널 방향으로 concat: [B, k*C, H, W]
            nn.Conv2d(obs_channels * context_frames, hidden_dim, 3,
            nn.SiLU(),
            nn.Conv2d(hidden_dim, hidden_dim, 3, stride=2, padding=
            nn.SiLU(),
            nn.Conv2d(hidden_dim, hidden_dim, 3, padding=1),
        )

        # 행동 인코더: k개의 과거 행동을 임베딩
        self.action_encoder = nn.Sequential(
            nn.Linear(action_dim * context_frames, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
        )

        # 시간 임베딩
        self.time_embed = nn.Sequential(
            SinusoidalPosEmb(time_embed_dim),
            nn.Linear(time_embed_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
        )
```

```python
        # 노이즈 예측 네트워크 (간소화된 ConvNet)
        # 입력: 노이즈가 추가된 다음 프레임 + 조건
        self.denoiser = nn.Sequential(
            nn.Conv2d(obs_channels + hidden_dim, hidden_dim * 2, 3,
            nn.SiLU(),
            nn.Conv2d(hidden_dim * 2, hidden_dim * 2, 3, padding=1),
            nn.SiLU(),
            nn.Conv2d(hidden_dim * 2, hidden_dim, 3, padding=1),
            nn.SiLU(),
            nn.Conv2d(hidden_dim, obs_channels, 3, padding=1),  # 원
        )

    def forward(
        self,
        noisy_next_obs: torch.Tensor,   # [B, C, H, W] 노이즈가 추가
        timestep: torch.Tensor,          # [B] 디퓨전 시간 스텝
        past_obs: torch.Tensor,          # [B, k, C, H, W] 과거 k프레
        past_actions: torch.Tensor,      # [B, k, action_dim] 과거 k
    ) -> torch.Tensor:
        """
        노이즈 예측: ε_θ(o_{t+1}^noisy, t_diff, o_{t-k:t}, a_{t-k:t}
        """
        B, k, C, H, W = past_obs.shape

        # 1. 과거 프레임 인코딩: [B, k*C, H, W] → [B, hidden_dim, H/2
        past_obs_flat = past_obs.view(B, k * C, H, W)
        context_feat = self.context_encoder(past_obs_flat)  # [B, h

        # 컨텍스트를 원래 해상도로 업샘플
        context_feat = F.interpolate(context_feat, size=(H, W), mode

        # 2. 행동 인코딩: [B, k*action_dim] → [B, hidden_dim]
        past_actions_flat = past_actions.view(B, -1)
        action_feat = self.action_encoder(past_actions_flat)  # [B,

        # 3. 시간 임베딩: [B] → [B, hidden_dim]
        t_feat = self.time_embed(timestep)  # [B, hidden_dim]

        # 4. 행동 + 시간 정보를 공간적으로 브로드캐스트하여 컨텍스트에 
        cond = action_feat + t_feat                              # [
        cond = cond[:, :, None, None].expand(-1, -1, H, W)     # [B
        context_feat = context_feat + cond                       # [

        # 5. 노이즈가 추가된 다음 프레임 + 조건 결합
        x = torch.cat([noisy_next_obs, context_feat], dim=1)   # [B

        # 6. 노이즈 예측
        noise_pred = self.denoiser(x)  # [B, C, H, W]

        return noise_pred
```

```python
# --- World Model 학습 예시 ---
def train_world_model_step(
    model: SimplifiedWorldModel,
    optimizer: torch.optim.Optimizer,
    past_obs: torch.Tensor,       # [B, k, C, H, W]
    past_actions: torch.Tensor,   # [B, k, action_dim]
    next_obs: torch.Tensor,       # [B, C, H, W] 실제 다음 프레임 (gr
    alpha_bar: torch.Tensor,      # 미리 계산된 DDPM 스케줄
    num_steps: int = 1000,
):
    """World Model의 한 학습 스텝.

    "이 과거 장면에서 이 행동을 하면, 다음 장면은 어떻게 보이는가?"를 학
    """
    B = next_obs.shape[0]
    device = next_obs.device

    # 1. 랜덤 시간 스텝
    t = torch.randint(0, num_steps, (B,), device=device)

    # 2. 다음 프레임에 노이즈 추가
    epsilon = torch.randn_like(next_obs)
    ab_t = alpha_bar[t].view(B, 1, 1, 1)
    noisy_next = torch.sqrt(ab_t) * next_obs + torch.sqrt(1 - ab_t)

    # 3. 노이즈 예측
    noise_pred = model(noisy_next, t, past_obs, past_actions)

    # 4. 손실 계산
    loss = F.mse_loss(noise_pred, epsilon)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    return loss.item()
```

C.7 HPT (Heterogeneous Pre-trained Transformers)

📄 *Limoyo et al. (MIT), "Learning a Universal Policy from Heterogeneous Robot Data", 2024*

> 💡 **왜 로봇에 "Foundation Model" 패러다임이 필요한가?** NLP에서는 GPT/BERT가 대량의 텍스트로 사전학습한 뒤, 소량의 태스크별 데이터로 fine-tuning하여 다양한 태스크에서 뛰어난 성능을 보인다. 로봇 학습에서도 같은 패러다임을 적용하고 싶지만, 핵심 장벽이 있다: **로봇마다 하드웨어가 다르다**. 7축 로봇팔, 모바일 로봇, 듀얼 암, 다관절 손은 관측 공간(카메라 수, 해상도, 센서 유형)과 행동 공간(자유도 수, 관절 유형, 좌표계)이 모두 다르다. 텍스트는 어떤 언어든 "토큰의 시퀀스"라는 통일된 표현이 가능하지만, 로봇 데이터는 통일된 표현이 없다.

다양한 로봇 데이터를 통합하여 사전학습:

```
[다양한 로봇 데이터]
  ├─ 모바일 로봇 (관측: LiDAR + 카메라, 행동: 2D 속도)
  ├─ 7축 로봇팔 (관측: 손목 카메라, 행동: 7D 관절 토크)
  ├─ 듀얼 암 (관측: 스테레오 카메라, 행동: 14D 관절 각도)
  └─ 다관절 손 (관측: 촉각 센서, 행동: 24D 손가락 각도)
        ↓
[로봇별 어댑터] → [공통 토큰 공간으로 변환]
        ↓
[통합 Transformer 백본 사전학습]
  (모든 로봇 데이터에서 공유 표현 학습)
        ↓
[특정 로봇에 Fine-tuning]
  (어댑터만 교체, 백본은 유지)
```

LLM의 "대규모 사전학습 → 특정 태스크 Fine-tuning" 패턴을 로봇에 적용.

⚠️ **HPT의 핵심 도전과 해결**: 서로 다른 로봇은 관측 공간(카메라 수, 해상도)과 행동 공간(자유도, 관절 유형)이 다르다. HPT는 각 로봇 형태에 맞는 입출력 어댑터(adapter)를 두고, 중간의 Transformer 백본은 공유하는 구조로 이 문제를 해결한다. 이는 언어 모델에서 다국어 모델이 공유 표현을 학습하는 것과 유사하다.

HPT의 구조적 직관:

```
로봇 A의 관측 → [어댑터 A] ──→ 공통 토큰 ──→ [Transformer 백본] → 공통
로봇 B의 관측 → [어댑터 B] ──→ 공통 토큰 ──→ [    (공유)     ] → 공통 토
로봇 C의 관측 → [어댑터 C] ──→ 공통 토큰 ──→ [               ] → 공통 토
```

어댑터는 로봇별로 다르지만 (관측/행동 차원이 다르니까), Transformer 백본은 **모든 로봇이 공유**한다. 백본은 "물체를 잡는다", "장애물을 피한다" 같은 **로봇 형태에 독립적인 (embodiment-agnostic) 추상적 정책 지식**을 학습한다. 새로운 로봇이 추가되면, 해당 로봇의 어댑터만 학습하면 사전학습된 백본의 지식을 바로 활용할 수 있다.

Part D: 기타 응용 (Other Applications)

> 💡 **왜 Diffusion이 이미지/비디오를 넘어 확장되는가?** Diffusion 모델의 수학은 "연속 공간에서의 노이즈→데이터 변환"이라는 완전히 범용적인 프레임워크다. 생성 대상이 이미지 픽셀이든, 3D 좌표이든, 로봇 행동이든, 단백질 구조이든, 음악 스펙트로그램이든 — 연속 벡터로 표현할 수 있다면 Diffusion을 적용할 수 있다. 이 섹션에서는 Diffusion이 과학, 음악, 분자 설계로 확장된 사례를 다룬다.

D.1 과학: AlphaFold3

📄 *Abramson et al. (Google DeepMind), "Accurate structure prediction of biomolecular interactions with AlphaFold 3", Nature 2024*

> 💡 **왜 단백질 구조 예측에 Diffusion이 필요한가?** AlphaFold2는 단백질의 3D 구조를 예측하는 데 혁명적이었지만, **단일 구조만 예측**했다. 실제 단백질은 여러 가능한 구조(conformation) 사이를 오가며, 특히 다른 분자와의 상호작용에서 구조가 바뀐다. AlphaFold3는 Diffusion 모델을 사용하여 **구조의 분포**를 표현할 수 있게 되었다 — 로봇 학습에서 multimodal 행동 분포를 Diffusion으로 표현한 것과 정확히 같은 동기이다. 단백질이 취할 수 있는 여러 가능한 구조를 샘플링할 수 있다.

단백질 3D 구조 예측에 **Diffusion Model** 사용:

- 입력: 아미노산 시퀀스 + 분자 정보
- 출력: 3D 원자 좌표
- 방법: 원자 좌표를 잠재 공간에서 디퓨전으로 생성

이것은 Part B에서 다룬 3D 생성과 구조적으로 유사하다. 3D 생성에서 NeRF의 좌표를 최적화했듯, AlphaFold3에서는 원자의 3D 좌표를 Diffusion으로 생성한다. 조건이 "텍스트 프롬프트"에서 "아미노산 시퀀스"로 바뀌었을 뿐, 근본 원리는 동일하다.

D.2 음악 생성

> 💡 **음악 생성과 이미지 생성의 구조적 유사성** 음악을 Mel-Spectrogram으로 변환하면, 그것은 본질적으로 **2D 이미지**다 (x축 = 시간, y축 = 주파수, 밝기 = 에너지). 따라서 이미지 Diffusion의 기법(Latent Diffusion, U-Net, CFG)을 거의 그대로 적용할 수 있다. AudioLDM은 실제로 Stable Diffusion의 아키텍처를 오디오 Mel-Spectrogram에 적용한 것이며, VAE가 이미지 대신 스펙트로그램을 인코딩/디코딩하고, CLIP 대신 CLAP(Contrastive Language-Audio Pretraining)을 사용하는 것이 주요 차이점이다.

- 📄 **MusicLM** (Google): 텍스트→음악 — *Agostinelli et al., "MusicLM: Generating Music From Text", 2023*
- 📄 **AudioLDM**: 오디오를 Mel-Spectrogram의 잠재 공간에서 디퓨전으로 생성 — *Liu et al., "AudioLDM: Text-to-Audio Generation with Latent Diffusion Models", ICML 2023*
- 📄 **Stable Audio**: 텍스트→음악/효과음 — *Evans et al., "Stable Audio: Fast Timing-Conditioned Latent Audio Diffusion", 2024*

D.3 분자 설계

- **Diffusion으로 분자 구조 생성**: 3D 좌표 + 원자 유형을 동시에 생성
- 신약 개발, 재료 과학에 활용
- Equivariant Diffusion Models: 회전/병진에 불변인 디퓨전

> 💡 **왜 분자 생성에 "Equivariant" Diffusion이 필요한가?** 분자의 물리적 성질은 분자를 회전하거나 평행 이동해도 변하지 않는다 (물 분자를 90도 돌려도 여전히 물이다). 이것을 **SE(3) 대칭**이라 한다. 일반 Diffusion 모델은 이 대칭을 모르기 때

> 문에, 같은 분자를 다른 각도에서 표현한 것을 서로 다른 데이터로 취급한다. Equivariant Diffusion은 네트워크 구조 자체에 이 대칭을 내장하여, 회전/병진에 대해 **동변(equivariant)**하게 동작한다. 이것은 학습 효율을 극적으로 높이고, 물리적으로 유효한 분자만 생성되도록 보장한다. 이미지 생성에서 convolutional 구조가 "병진 등변"을 보장하는 것과 같은 원리의, 더 일반화된 버전이다.

📄 *Hoogeboom et al., "Equivariant Diffusion for Molecule Generation in 3D", ICML 2022*

전체 시리즈 요약 (Full Series Summary)

강의 1~12에서 배운 것

```
[기초 모델]                                    [현대 모델]
강의 1: VAE ──────────────────────────────→ LDM의 AutoEncoder
강의 2: VQ-VAE, Normalizing Flow ──────────→ VQ-GAN, CNF, Flow Matchi
강의 3: GAN ──────────────────────────────→ 판별자 학습, 적대적 훈련
강의 4: StyleGAN, VQ-GAN ─────────────────→ 2-Stage 생성의 기반
강의 5: DALL-E, Score-Based ──────────────→ 텍스트→이미지, 스코어 함수

[핵심 기술]
강의 7: DDPM ─────────────────────────────→ 모든 Diffusion의 기반
강의 8: CLIP, CFG ────────────────────────→ 조건부 생성의 핵심
강의 9: Latent Diffusion / SD ────────────→ 모든 것의 결합

[최신 발전]
강의 10: Flow Matching ───────────────────→ SD3, FLUX의 기반
강의 11: DiT, Consistency, VAR ───────────→ 최신 아키텍처
강의 12: 비디오, 3D, Physical AI ─────────→ 응용 확장
```

패러다임의 진화

```
2014        2020        2022        2023        2024        2025
 |           |           |           |           |           |
 VAE        DDPM      Stable      Flow        VAR      Hybrid
 GAN         |      Diffusion   Matching      DiT     AR+Flow
  |          |          |           |           |          |
  └── 기초 ──┴── 실용화 ┴── 대중화 ──┴── 효율화 ──┴── 융합 ──┘
```

핵심 통찰

모든 생성 모델은 결국 **같은 문제**를 푼다:

$$\text{간단한 분포 } p_{\text{init}} \xrightarrow{\text{학습된 변환}} \text{복잡한 분포 } p_{\text{data}}$$

방법	변환 방식
VAE	잠재 변수를 통한 간접 매핑
GAN	암시적 매핑 (판별자로 학습)
Normalizing Flow	가역 변환의 체인
DDPM/SDE	확률적 미분방정식 시뮬레이션
Flow Matching/ODE	결정론적 미분방정식 시뮬레이션
AR (VAR, MAR)	자기회귀적 순차 생성

어떤 방법을 선택하든, 충분한 데이터와 모델 크기가 있으면 좋은 결과를 얻을 수 있다. 현재의 트렌드는 이 방법들이 **수학적으로 통합되고, 실용적으로 결합**되는 방향으로 가고 있다.

> 💡 **이 강의 시리즈의 가장 중요한 메시지** 12개 강의를 관통하는 핵심 메시지는: **"생성 모델의 수학적 원리는 도메인에 독립적이다."** VAE의 잠재 공간 아이디어는 LDM의 핵심이 되었고, DDPM의 노이즈-제거 원리는 비디오, 3D, 로봇, 단백질, 음악 모두에 적용되었다. DiT의 Transformer 아키텍처는 Sora의 비디오 생성으로 확장되었다. 하나의 수학적 프레임워크를 깊이 이해하면, 그것이 다른 도메인으로 어떻게 전이되는지를 직관적으로 파악할 수 있다.
>
> 앞으로의 발전 방향은: (1) **통합** — Flow Matching + AR + Diffusion의 하이브리드 모델, (2) **효율화** — Consistency Models, Distillation 등으로 추론 속도 개선, (3) **범용화** — 하나의 모델이 이미지, 비디오, 3D, 오디오를 동시에 생성하는 "Unified Foundation Model"이다.

참고문헌 (References)

Video Generation

1. OpenAI, "Video generation models as world simulators", Technical Report, 2024.
2. Polyak, A., et al., "Movie Gen: A Cast of Media Foundation Models", Meta Technical Report, 2024.
3. Google DeepMind, "VEO 2 / VEO 3", 2024–2025.

3D Generation

4. Poole, B., Jain, A., Barron, J.T., & Mildenhall, B., "DreamFusion: Text-to-3D using 2D Diffusion", ICLR 2023.
5. Lin, C.-H., et al., "Magic3D: High-Resolution Text-to-3D Content Creation", CVPR 2023.
6. Wang, Z., et al., "ProlificDreamer: High-Fidelity and Diverse Text-to-3D Generation with Variational Score Distillation", NeurIPS 2023.

7. Liu, R., et al., "Zero-1-to-3: Zero-shot One Image to 3D Object", ICCV 2023.

Physical AI / Robotics

8. Chi, C., et al., "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion", RSS 2023.
9. Limoyo, O., et al. (MIT), "Learning a Universal Policy from Heterogeneous Robot Data", 2024.
10. Yang, M., et al., "Learning Interactive Real-World Simulators (UniSim)", ICLR 2024.
11. Bruce, J., et al., "Genie: Generative Interactive Environments", ICML 2024.
12. NVIDIA, "Cosmos World Foundation Models", Technical Report, 2024.

Science & Other Applications

13. Abramson, J., et al. (Google DeepMind), "Accurate structure prediction of biomolecular interactions with AlphaFold 3", Nature, 2024.
14. Agostinelli, A., et al., "MusicLM: Generating Music From Text", 2023.
15. Liu, H., et al., "AudioLDM: Text-to-Audio Generation with Latent Diffusion Models", ICML 2023.
16. Evans, Z., et al., "Stable Audio: Fast Timing-Conditioned Latent Audio Diffusion", 2024.
17. Hoogeboom, E., et al., "Equivariant Diffusion for Molecule Generation in 3D", ICML 2022.